API SECURITY RISK ANALYSIS REPORT

FUTURE INTERNS – CYBER SECURITY TASK 3

PREPARED BY:
SHUSEEL BARAL

March, 2026

1. Executive Summary

This report outlines the findings from a targeted security assessment of the JSON Placeholder API. The objective of this assessment was to evaluate authentication controls, authorization enforcement, data exposure risks, and potential abuse vectors. The assessment utilized Postman testing, evaluated against industry-standard API security checklists. The testing was conducted using ethical, non-invasive methods in accordance with responsible security practices.

The current API implementation for **https://jsonplaceholder.typicode.com** lacks fundamental security gates. While functional for testing, deploying this architecture in a production environment presents a high-risk profile due to the absence of authentication and predictable resource identifiers.

2. Objective

This task's objectives are to conduct a read-only security risk analysis of a public API, find common security flaws in the API, and offer precise remedial suggestions in clear business language.

3. Scope and Methodology

Scope:

The assessment was limited to publicly accessible endpoints of the JSON placeholder API. Only safe GET and regulated POST requests were carried out.

Methodology:

The assessment followed a structured security review process:

- Documentation review
- Endpoint enumeration
- Authentication verification
- Authorization testing
- Response analysis
- Header inspection.

4. Tools Used

- Postman
- Browser Developer Tools
- Manual HTTP analysis

5. API Tested

API Name:

JSON Placeholder Base URL: https://jsonplaceholder.typicode.com

Endpoints Tested:

- /posts
- /comments
- /albums
- /photos
- /todos
- /users

6. Risk Classification Criteria

The identified risks in this analysis are classified as follows.

- **High:** Immediate risk of data compromise or unauthorized modification.
- **Medium:** Data exposure or misuse possible with moderate impact.
- **Low:** Minimal business impact or limited exploitability.

7. Risk Summary

Risk Identified	Severity	Business Impact
Unauthenticated Access	High	Unauthorized users can access, create, and modify data across all endpoints.
Broken Object Level Authorization (BOLA)	High	Attackers can manipulate IDs in the URL to access or modify data belonging to any user.
Excessive Data Exposure	Medium	Sensitive user details (names, emails, addresses) are returned in full in the /***users*** endpoint.
Missing Rate Limiting	Medium	Vulnerability to Denial of Service (DoS) or automated scraping attacks.
Security Header Deficiencies	Low	Increased susceptibility to Clickjacking and Man-in-the-Middle (MITM) attacks.

8. Detailed Risk Analysis

8.1. Unauthenticated Endpoints

The resources can be fetched, created, updated, and deleted without any authentication headers.

- **Observation:** There is no requirement for standard authentication tokens.
- **Risk:** This allows any external entity to interact with the API resources, leading to potential data corruption or unauthorized resource consumption.

Proof of concept:

Test for Broken Authentication: Send a GET request to each endpoint (e.g., /posts) without any Authorization header.

Identified Risk: API returns 200 OK instead of 401 Unauthorized; the endpoint is unauthenticated.

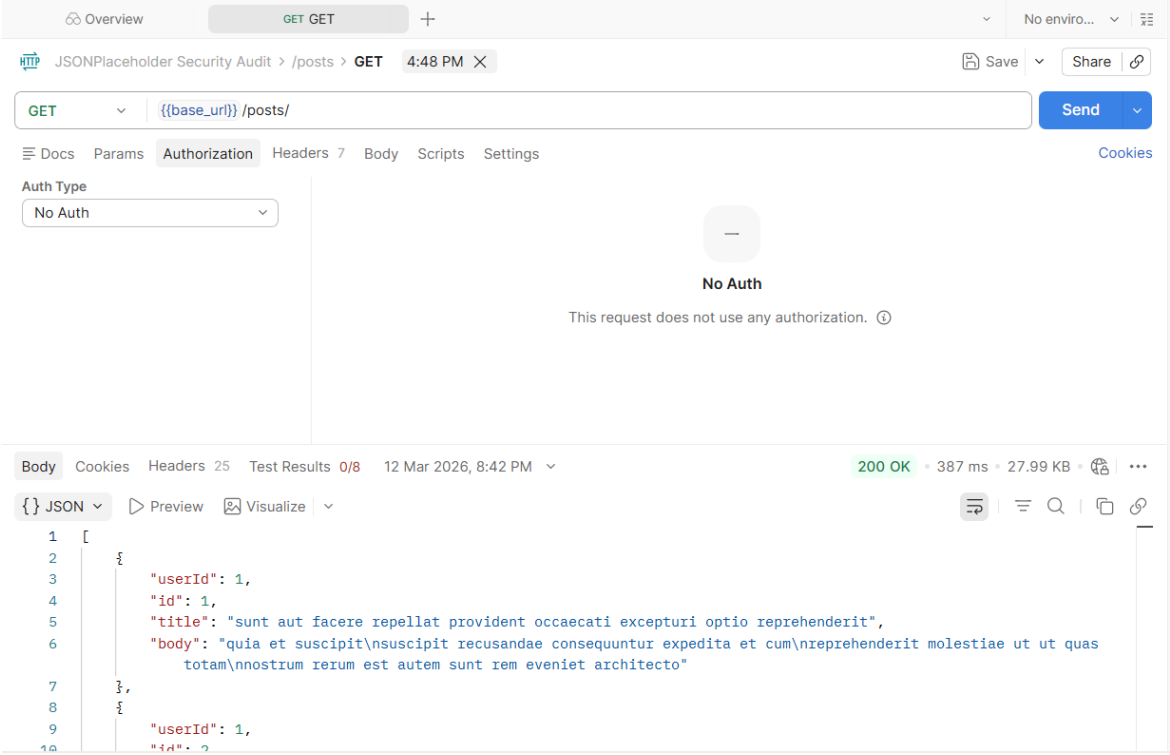


Fig. 1: API returns 200 OK instead of 401 Unauthorized

8.2. Broken Object Level Authorization (BOLA)

The API relies heavily on predictable, auto-incremented integer IDs in the URL path (e.g., /posts/1).

- **Observation:** The API allows filtering and accessing resources simply by changing the ID or userId parameter.
- **Risk:** The use of auto-incremented IDs rather than UUIDs makes it easy for attackers to enumerate and access all records in the database.

Proof of concept:

Test for Broken Object Level Authorization (BOLA): Attempted to access a specific resource (e.g., GET /posts/1) and then manually changed the ID to GET /posts/2.

Observed that the API allows access to any ID without checking whether the requester owns that resource.

Identified Risk: While verifying the resource identifiers by inspecting the response body. IDs are auto-incrementing integers (1, 2, 3...) rather than UUIDs; the API is vulnerable to resource enumeration.

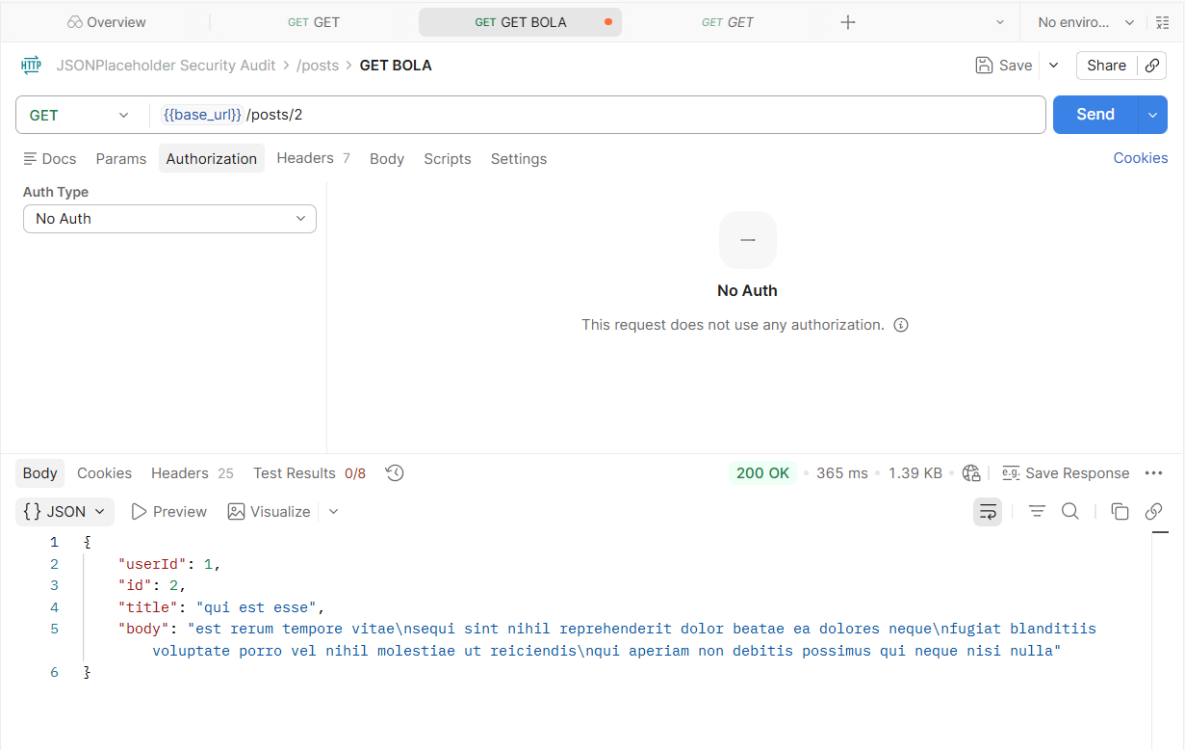


Fig. 2: API allows access to any ID without checking

8.3. Excessive Data Exposure

The /users endpoint returns complete objects that may include more information than the client requires.

- **Observation:** Response payloads include specific IDs and user-linked data that could be used for further social engineering or targeted attacks.
- **Risk:** Returning sensitive data like credentials or overly specific user info violates privacy standards.

Proof of concept:

Inspected Payload Content: Reviewed responses (especially /users) for sensitive data like name, username, email, etc.

Identified Risk: Exposing sensitive data, like name, username, and email, that violates privacy standards.

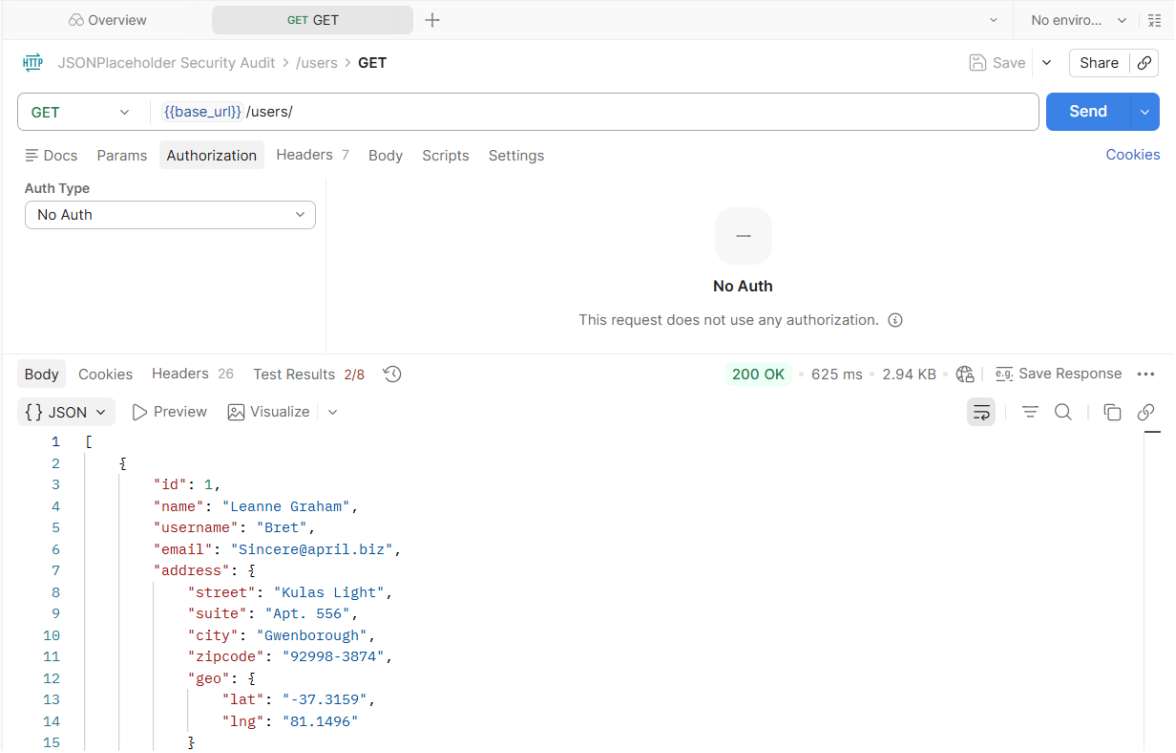


Fig. 3: Exposing sensitive data, such as name, username, email, etc.

Information Exposed in Header: Found unwanted information on headers like X-Powered-By, Server, etc.

Identified Risk: These "fingerprinting" headers revealed implementation details that help attackers tailor their exploits.

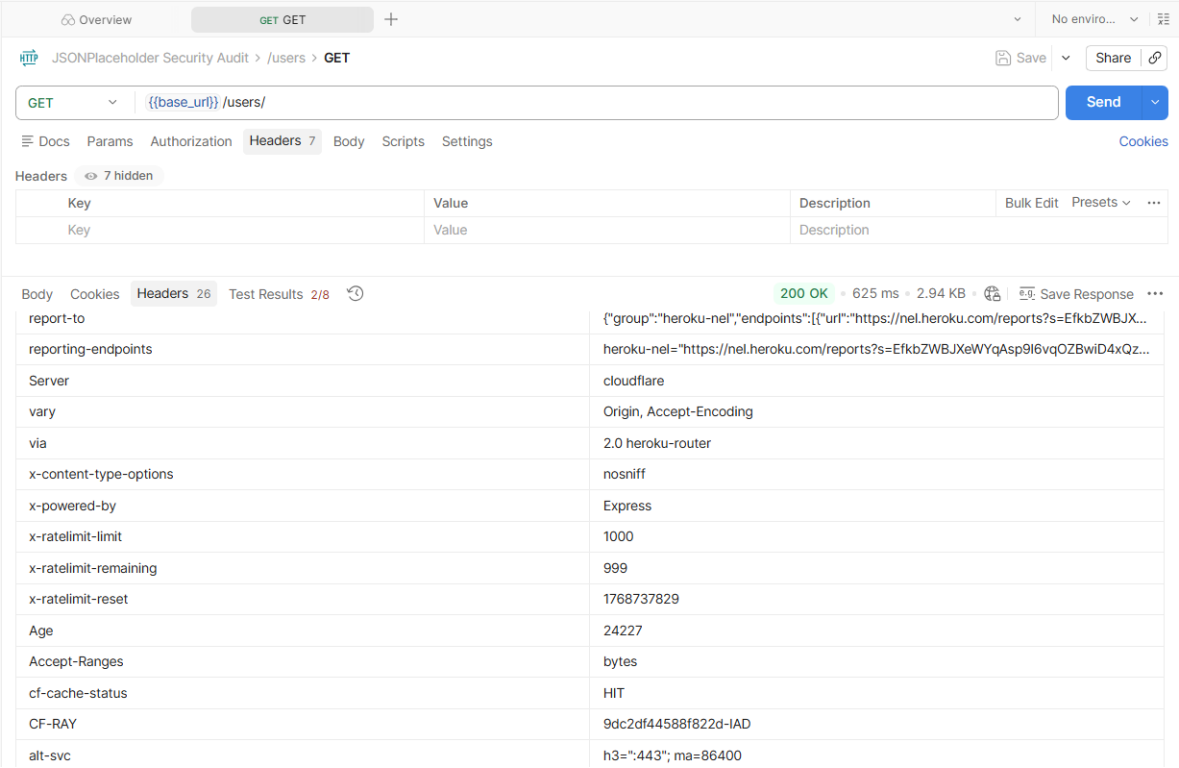


Fig. 4: Found unwanted information on headers

8.4. Missing Security Headers

The standard security headers, such as “X-Frame-Options” and “Content-Security-Policy”, are absent from the typical response flow.

- **Observation:** There is no mention of X-Frame-Options: deny or Content-Security-Policy in the API's standard response structure.
- **Risk:** Without standard security headers, the API is vulnerable to Clickjacking and Man-in-the-Middle (MITM) attacks.

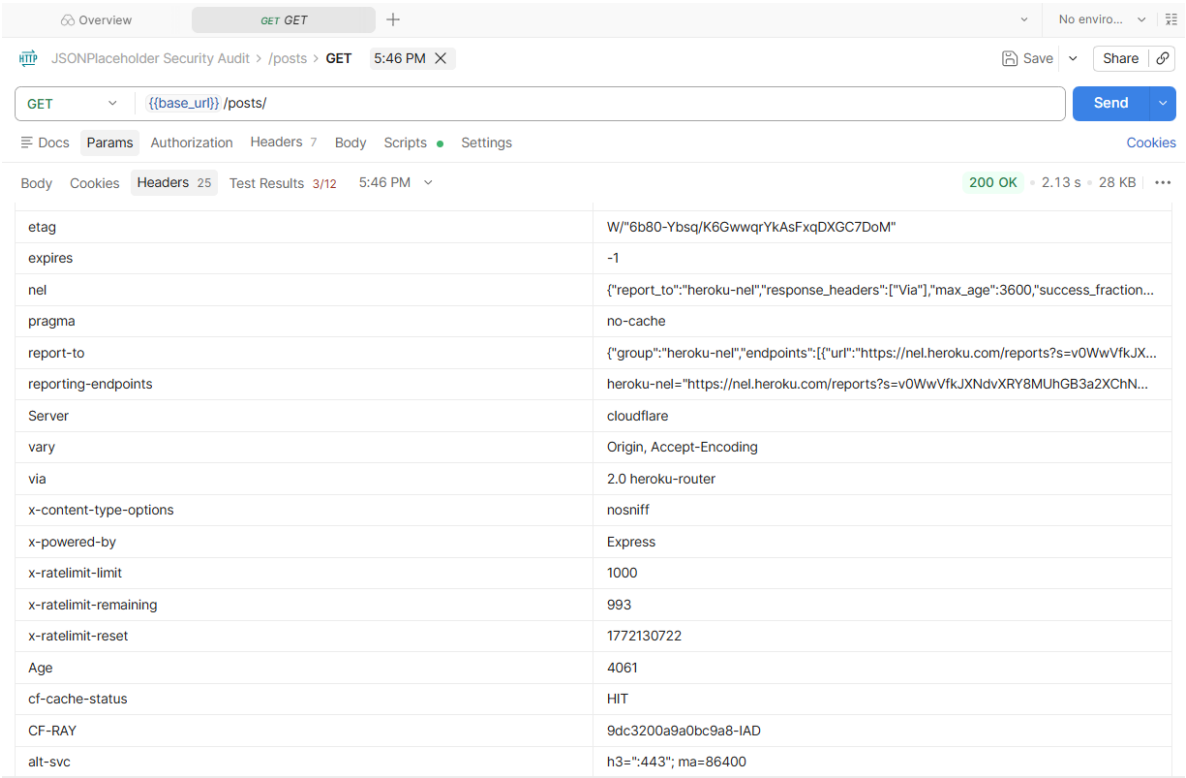
Proof of concept:

Inspected the "Headers" tab in the Postman response pane to verify defensive configurations.

Security Header Audit: Not found the following mandatory headers:

- X-Frame-Options: deny: Prevents clickjacking.
- Content-Security-Policy: default-src 'none': Restricts where resources can be loaded from.

Identified Risk: The security headers are not present in the API.



etag	W/"6b80-Ybsq/K6GwwqrYkAsFxqDXGC7DoM"
expires	-1
nel	{"report_to":"heroku-nel","response_headers":["Via"],"max_age":3600,"success_fraction...
pragma	no-cache
report-to	{"group":"heroku-nel","endpoints":[{"url":"https://nel.heroku.com/reports?s=v0WwVfkJX...
reporting-endpoints	heroku-nel="https://nel.heroku.com/reports?s=v0WwVfkJXNdvXRY8MUhGB3a2XChN...
Server	cloudflare
vary	Origin, Accept-Encoding
via	2.0 heroku-router
x-content-type-options	nosniff
x-powered-by	Express
x-ratelimit-limit	1000
x-ratelimit-remaining	993
x-ratelimit-reset	1772130722
Age	4061
cf-cache-status	HIT
CF-RAY	9dc3200a9a0bc9a8-IAD
alt-svc	h3=":443"; ma=86400

Fig. 5: The security headers are not present in the API

8.5. Missing Rate Limiting

There are no apparent Rate Limit policies (such as Quota, Spike Arrest, or Concurrent Rate Limit) implemented to avoid DDoS or brute-force attacks.

- **Observation:** No apparent Rate Limit policies are implemented.
- **Risk:** This allows DDoS or brute-force attacks.

Proof of concept:

Brute-Force Simulation: Used the Postman Runner to send 100 rapid GET requests to /posts. Expected the response for a 429 Too Many Requests status code.

Identified Risk: The API continues to return 200 OK, it lacks rate limiting (Throttling), and is vulnerable to DDoS or scraping.

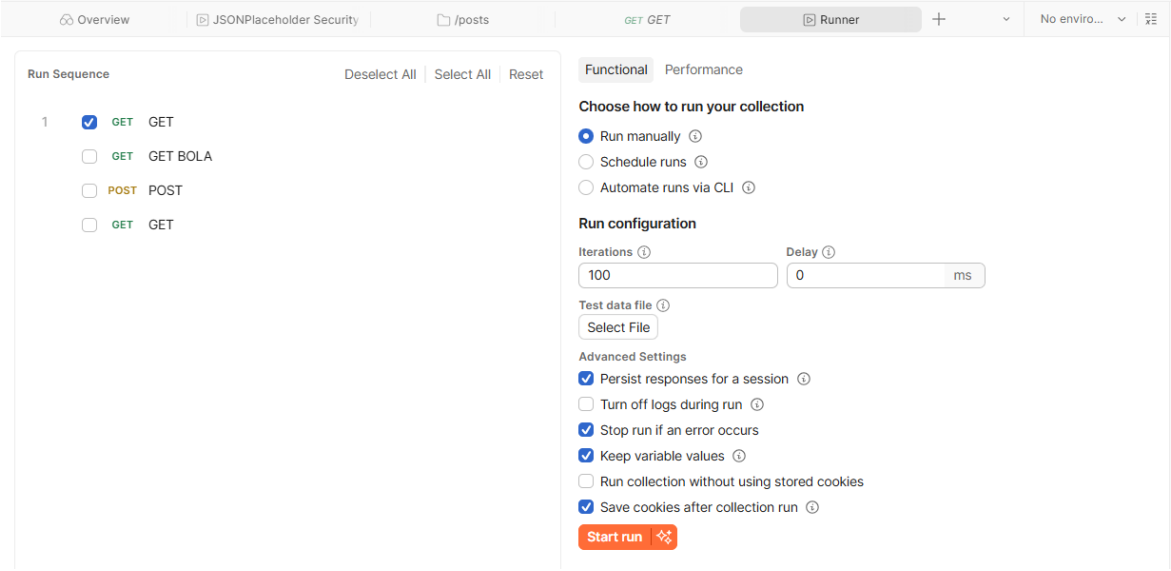


Fig. 6: Runner Setup to send 100 rapid GET requests

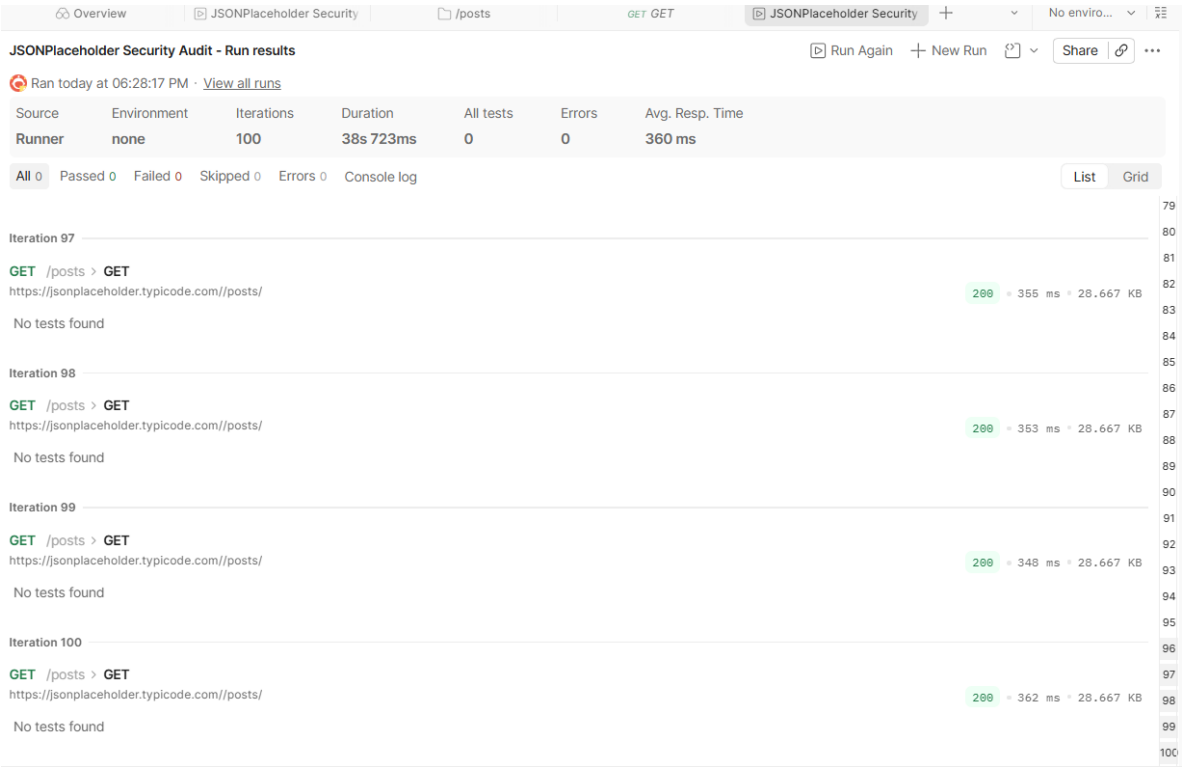


Fig. 7: Runner result after sending 100 rapid GET requests

9. Business Impact of Inaction

Maintaining the current unauthenticated state carries the following significant business risks:

- **Data Integrity:** Without proper HTTP method validation and authentication, any user can theoretically modify or delete resources.
- **Privacy Compliance:** Excessive data exposure (e.g., full user details in responses) can lead to regulatory fines under frameworks like GDPR or CCPA.
- **Reputational Damage:** A successful "Billion Laughs" or XML bomb attack through unvalidated entity expansion could lead to prolonged service outages.

10. Remediation Suggestion

To fix these vulnerabilities, the following architectural changes should be prioritized:

- **Implement Standard Authentication:** Transition from open access to a standard authentication model (e.g., OAuth2) and ensure all endpoints are protected.
- **Enforce Authorization Logic:** Use a `/me/` endpoint structure for user-specific data and validate that the authenticated user has permission to access the requested resource ID.
- **Adopt UUIDs:** Replace auto-incrementing integer IDs with UUIDs to prevent resource enumeration attacks.
- **Apply Rate Limiting:** Deploy an API Gateway to implement Throttling, Quotas, and Spike Arrest policies to prevent abuse.
- **Harden Headers:** Configure the server to send mandatory security headers, including X-Frame-Options: deny and Content-Security-Policy: default-src 'none'.
- **Input Validation:** Rigorously validate the Content-Type of incoming requests and sanitize all user inputs to prevent SQL injection and XSS.

11. Strategic Cost-Benefit Analysis

Security Control	Implementation Cost	Benefit / Risk Mitigation
Standard Authentication (OAuth)	**Medium:** Requires identity provider setup and token management.	**High:** Prevents unauthorized access and protects endpoints from broken authentication processes.
Rate Limiting & Throttling	**Low:** Usually handled at the API Gateway level.	**High:** Mitigates DDoS attacks, brute-force attempts, and resource exhaustion.
UUIDs instead of Integer IDs	**Medium:** Requires database schema changes and logic updates.	**Medium:** Prevents easy resource enumeration and Broken Object Level Authorization (BOLA).
Security Headers	**Low:** Simple configuration change on the server side.	**Medium:** Protects against SSL Strip, Clickjacking, and MIME-sniffing attacks.

12. Conclusion

The API security assessment identified multiple control gaps that would pose significant security risks in a real-world SaaS deployment. The absence of authentication, authorization enforcement, and rate limiting represents critical weaknesses. Implementing secure API design principles would substantially improve the overall security posture.